

Full Stack Mobile Application

Design, build, deploy and defend a small mobile application on your own — React Native on the front, Node and MongoDB behind it, running on a live host.

Weight 20%	Marked out of 100 (scaled)	Work Individual	Duration 8 weeks
----------------------	--------------------------------------	---------------------------	----------------------------

1 Overview

You will build and host one working mobile application end to end. Everything in it is yours: the schema, the API, the screens, the deployment, and the explanations you give at the viva.

Frontend	React Native
Backend	Node.js + Express.js
Database	MongoDB (Atlas)
Hosting	Render, Railway, AWS, DigitalOcean or similar

You may choose your own system title and domain, but **the topic must be approved** before you start building. Keep it small. A narrow app that works, is hosted and is well understood scores far higher than an ambitious one that half runs on localhost.

2 What your system must contain

Authentication

- User registration and login
- Password hashing (bcrypt)
- JWT-based authentication
- Protected routes on the backend, and a protected area in the app

Two entities

Your system needs a **primary entity** and a **related entity** that references it — that relationship is what makes this a system rather than a list. Users do not count as one of the two.

- **Primary entity:** full CRUD (create, read all, read one, update, delete), plus one image upload
- **Related entity:** full CRUD, with at least one reference field pointing at the primary entity and at least one status or state change
- At least one piece of real business logic — a rule the database cannot enforce on its own, such as blocking a booking when capacity is full, or recalculating a total when an item changes

Hosting

- Backend deployed and reachable over the internet
- MongoDB Atlas connected from the deployed backend
- The mobile app points at the hosted API, not at your machine
- **A localhost demo is not accepted at the final evaluation**

3 Scope boundaries

These are deliberately out of scope. Adding them will not earn extra marks, and it will cost you the time you need for hosting and testing.

- More than two entities beyond User
- Payment gateways, SMS or email delivery, push notifications
- Real-time features (sockets, live chat, live tracking)
- Machine learning, recommendations or analytics dashboards
- Web admin panels — administration happens inside the mobile app
- CI/CD pipelines, containers, automated test suites

Roles are optional. If you do add an admin role, one boolean field and one middleware check is enough.

4 Technical requirements

Backend

- RESTful API design
- Sensible folder structure: routes, controllers, models, middleware
- Middleware used for authentication and for file upload
- Error handling, with correct HTTP status codes
- Input validation before anything reaches the database

Mobile

- Working navigation between screens
- Functional components and hooks
- Form validation with visible error messages
- Loading and empty states
- All data fetched from your API — **no hardcoded data**

5 Deliverables

- 1 **Source code** in a single Git repository, with commits spread across the eight weeks. One large commit at the end is treated as a warning sign.
- 2 **Live backend URL** and the `.env` variable names you configured (values redacted).
- 3 **Report** (8–12 pages) containing the items below.
- 4 **Demonstration** of the app running against the deployed backend.

The report must include a problem statement, a system architecture diagram, a database schema diagram, an API endpoint table, a short deployment walkthrough, and a one-page reflection on what broke and how you fixed it.

6 Rules

- You must be able to explain any line of your own code. If you cannot, marks are reduced regardless of whether the feature works.
- Plagiarism, including copied projects and shared repositories, is awarded zero.
- AI tools are allowed for learning support, not for generating your system. Declare what you used and where.
- Firebase-only projects are not accepted. The backend must be Node + MongoDB.
- The topic must be approved before development begins.

7 Suggested topics

Each of these reduces cleanly to two entities. Pick one, or propose your own with the same shape.

System	Primary entity	Related entity
Hostel room booking	Room	Booking
Clinic appointments	Doctor	Appointment
Food ordering	Menu item	Order
Event booking	Event	Ticket
Fitness tracking	Workout plan	Session log
Campus lost & found	Item	Claim

You may reuse the domain of your ITP or ML&AI project, but the code you submit here must be written by you for this module.

8 Worked example — hostel room booking

A reference for the expected size of the system, not a required topic.

Entity 1 — Room (primary)

roomNumber, roomType (Single / Double / Triple), pricePerMonth, capacity, currentOccupancy, description, image, availabilityStatus

Create, view all, view one, update, delete. Image uploaded with Multer, file type and size validated, stored on the server or a cloud bucket, and the resulting URL returned to the app.

Entity 2 — Booking (related)

userId → User, roomId → Room, bookingDate, startDate, endDate, status (Pending / Approved / Rejected)

Create a request, view your booking history, approve or reject, cancel.

Business logic tying them together

- Approving a booking increments `currentOccupancy`
- A room at full capacity flips `availabilityStatus` and refuses new requests
- Cancelling an approved booking releases the place again

Three things carry most of the marks in a system this size: the reference between the two entities, the rule that keeps occupancy honest, and the fact that it all runs against a hosted API. Build those first.

9 Eight-week schedule

A guide, not a submission requirement. Deploying in week 3 rather than week 8 is the single most useful thing on this list.

Week	Focus
1	Topic approval, schema design, repository set up
2	Authentication: registration, login, hashing, JWT, protected routes
3	Primary entity CRUD; deploy the backend early, even while incomplete
4	React Native navigation, auth screens, first list and detail screens
5	Related entity CRUD and the business logic between the two
6	Image upload end to end, form validation, error and empty states
7	Testing against the hosted API, fixing what only worked locally
8	Report, diagrams, demo rehearsal, viva preparation

10 Marking

A. Code and technical implementation — 40 marks

Assessed from the repository, the live system and hands-on testing.

Area	What is assessed	Marks
Authentication	Registration, login, hashing, JWT, protected routes	8
Entity CRUD and logic	Both entities complete, referenced correctly, business rule enforced	12
Mobile application	Navigation, hooks, validation, API integration, no hardcoded data	10
Image upload	Upload, validation, storage, URL returned and rendered	4
Deployment	Backend live, Atlas connected, app using the hosted API	4
Code quality and Git	Structure, error handling, status codes, commit history	2
Total		40

B. Viva — 60 marks

Criterion	Description	Marks
Explaining your implementation	Can walk through logic, functions and request flow clearly	20
System design decisions	Why the schema, routes and structure are the way they are	10
Backend and database concepts	Schemas, references, controllers, middleware, status codes	10
Mobile and API interaction	Request flow, state, token handling, UI behaviour	10
Problem solving and debugging	Answers “what if” questions and live scenario changes	10
Total		60

Attendance at the viva is compulsory. A working system you cannot explain is marked as if it were not yours.